# Patch-based Microwave Link Attenuation

This project computes **rain-induced microwave link attenuation** using OPERA rainfall patches and 4TU link data, following ITU-R P.838-3.

The code is modular (each file $\leq$ 200 lines) and designed for **debuggable, patch-by-patch processing**.

---

## What the program does

For each selected rainfall patch: 1. Load rainfall from the patch's HDF5 file (`source_file`). 2. Crop rainfall to the patch extent. 3. Replace NaN / missing values with 0. 4. Refine rainfall from **2 km → 125 m** (16×16 inheritance). 5. Smooth refined rainfall using a **Gaussian filter** (σ = 1 refined pixel, `mode="nearest"`). 6. Translate 4TU links into the patch domain using a fixed anchor. 7. Keep links whose **both endpoints lie inside the patch**. 8. Compute per-link rain attenuation using **ITU-R P.838-3**. 9. Write one JSONL file per patch with attenuation results.

Optionally, the program can output a **pixel-by-pixel attenuation trace** for a chosen link.

---

## Directory structure

```
project/
├── main.py                    # CLI entry point
├── rainfall_processing.py     # H5 loading, crop, refine, smooth
├── link_geometry.py           # Projection, translation, link filtering
├── intersection.py            # Link-pixel intersection lengths
├── itu_r_p8383.py              # ITU-R P.838-3 model (copied)
├── itu_model.py               # Thin ITU wrapper
├── attenuation.py             # Attenuation + outputs
└── README.md
```

---

## Required inputs

### 1. Patch list JSONL

Each record must contain at least: - `patch_id` (string) - `source_file` (full path to HDF5 file) - `x_min`, `x_max`, `y_min`, `y_max` (crop indices in OPERA grid) - `patch_lon_min`, `patch_lon_max` - `patch_lat_min`, `patch_lat_max` - `width_km`, `height_km`

### 2. Patch attributes JSONL

- Contains records keyed by `patch_id`.
- A patch is **selected** if it appears here.

### 3. 4TU links JSONL

Each record must contain: - `XStart`, `YStart` (lon, lat) - `XEnd`, `YEnd` (lon, lat) - `Frequency` (GHz) *(optional; defaults to 10 GHz)* - `Polarization` *(optional; defaults to user choice)*

---

## Running the program

```
python main.py
```

You will be prompted for: - patch list JSONL path - patch attributes JSONL path - 4TU links JSONL path - output directory - number of patches `k` - default polarization if missing `(H/V)` - optional debug settings

All prompts repeat until valid input is given.

---

## Outputs

### Per-patch output

For each processed patch:

```
patch_<patch_id>.jsonl
```

Each record corresponds to one link and contains: - all original link fields - `orig_index` (index in original link list) - `freq_ghz`, `pol` - `link_index` (index within this patch output) - `attenuation_db`

### Debug output (optional)

If debug mode is enabled:

```
debug_<patch_id>_link<link_index>.json
```

Contains: - list of intersected refined pixels - rainfall per pixel (mm/h) - segment length per pixel (m, km) - ITU specific attenuation (dB/km) - per-pixel and cumulative attenuation (dB) - total attenuation and path length

This is intended for **sanity checking and validation**.

## Coordinate conventions

- Rainfall grids: derived from OPERA HDF5, indexed as `[i, j]` with
- `i` increasing southward
- `j` increasing eastward
- Geometry calculations: **EPSG:28992 (RD New)** in meters
- Link lengths converted to **km** before ITU attenuation

---

## Notes

- If frequency or polarization is missing in the link JSON, the program prints a warning and uses defaults.
- Gaussian smoothing uses `mode="nearest"` to match the existing benchmark implementation.
- The intersection algorithm assumes both link endpoints lie inside the patch rectangle.

---

## Typical workflow

1. Run with a small `k` (e.g. 1–3 patches).
2. Enable debug mode for a specific patch + link index.
3. Inspect the debug JSON to verify rainfall, geometry, and attenuation.
4. Scale up to more patches once validated.